```
const axios = require('axios');

const fetchMovies = async (url) => {
  const response = await axios
    .get(`${url}/movies`)
    .then((res) => res.data)
    .catch((err) => err.response);
  return response;
};

...

const deleteMovie = async (url, id) => {
  const response = await axios
    .delete(`${url}/movie/${id}`)
    .then((res) => res.data.message)
    .cath((err) => err.response.data.message);
  return response;
};
```

The `fetchMovies` function makes a `GET` request to the URL passed in, followed by `'/movies'`. This should return the response from the `GET` request. If there are any errors, the error message is returned instead. On the other hand, the `deleteMovie` function makes a `DELETE` request to the URL passed in, followed by `'/movie/${id}'` where the `${id}` variable corresponds to the movie ID. If the movie is found, this should return a message that the movie has been deleted. If the movie doesn't exist, the `DELETE` request should return a message that the movie isn't found. Now that you have an overview of both the consumer and provider, let's get started with writing our first consumer contract test!

4.3.3 Consumer contract

If you remember from chapter 2, when starting with contract testing, especially consumer-driven contract testing (CDCT), the consumer should specify the contract. Let's imagine that the following scenarios are expected from the movies API, which forms the basis of the consumer contract:

1. **Scenario: Get all movies**

    Given there is a movies API

    When the consumer fetches all movies

    Then the movies API should return a status code of 200